Universidad Central de Venezuela

Facultad de Ciencias

Escuela de Computación

Sistemas Operativos (6004)

Semestre I-2026



PreLaboratorio 7: Contenerización y Mecanismos de Aislamiento del Kernel (Docker)

P-Torrealba MIguel

sección: C2

sección lab: 01 (P: Guillermo H)

Javier Alessandro Ramírez Amaro

C.I. 31.707.800

Enlace de github:
https://github.com/JAVIERRAMIREZ1212/lab7-so-docker-31.707.800

Marco teórico:
1) Hipervisor vs contenedores linux

- **Hipervisor:** capa que virtualiza hardware. Cada máquina virtual (VM) cree tener CPU, memoria, discos, red, interrupciones, etc. Por eso necesita un kernel invitado (Guest OS) propio.
- **Contenedor Linux:** no virtualiza hardware. Aísla procesos dentro del mismo kernel del host usando namespaces, cgroups, capabilities, seccomp, LSM y sistemas de archivos de unión. Por eso comparte el kernel con el host.

Tipo 1
El hipervisor corre directamente sobre el hardware, sin un sistema operativo anfitrión intermedio.

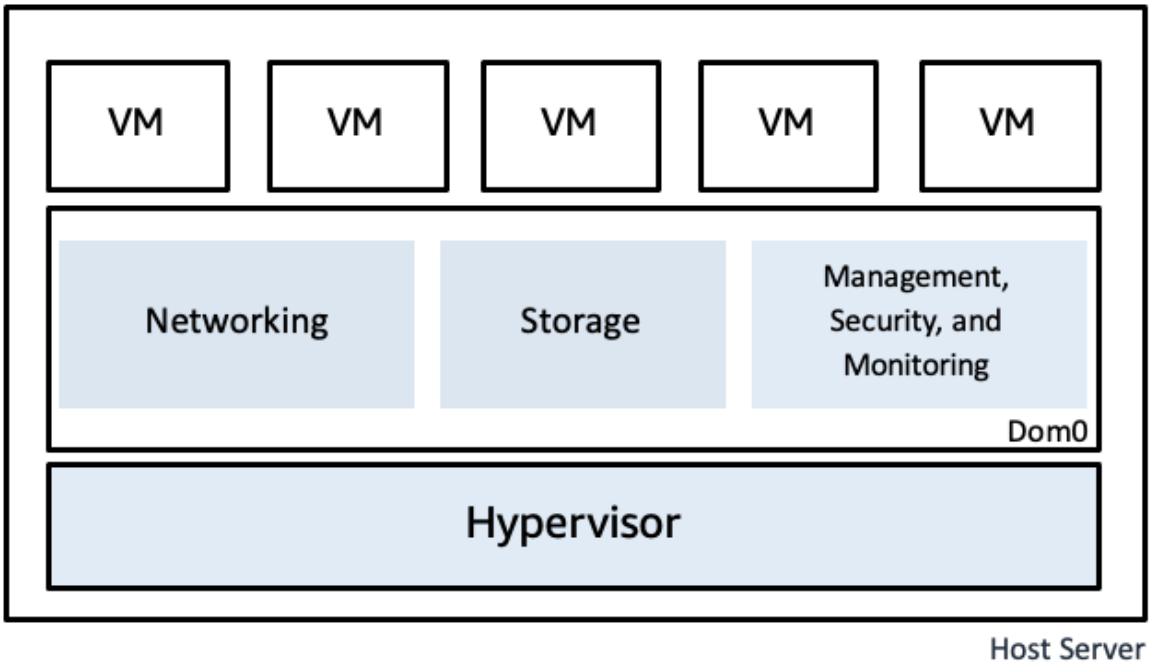


imagen sacada de la fuente ¿Cuál es la diferencia entre los hipervisores de tipo 1 y de tipo 2?

El hipervisor gestiona el cpu, memoria, interrupciones y dispositivos y utiliza técnicas como Trap-and-emulate (intercepta instrucciones privilegiadas del Guest OS), Binary translation (traduce instrucciones conflictivas), Paravirtualización (el Guest OS sabe que está virtualizado y usa hipercalls) y Virtualización asistida por hardware (Intel VT-x, AMD-V, EPT/NPT). Cada VM ejecuta su propio kernel, sus drivers, su scheduler, su gestor de memoria. además de tener un rendimiento muy bueno, pero hay overhead por VM exits, emulación de dispositivos, interrupciones virtuales y paginación anidada.

Tipo2

El hipervisor orre como una aplicación sobre un sistema operativo anfitrión.

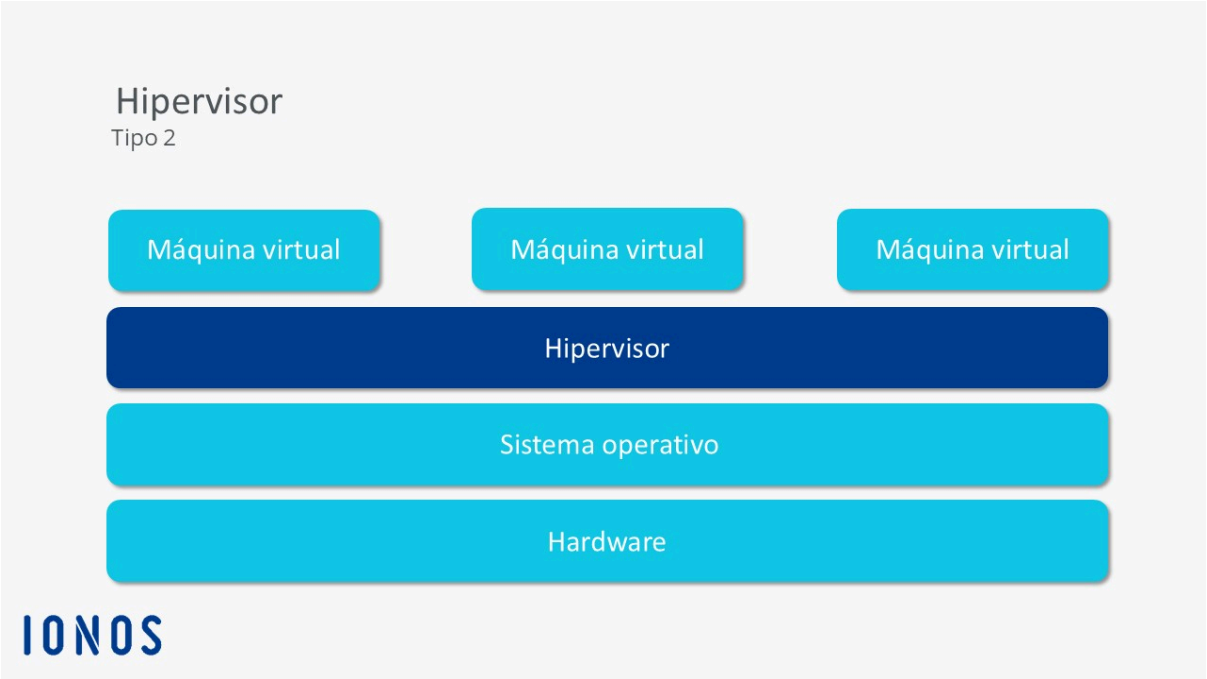


Imagen sacada de la fuente [Hypervisor - ¿Qué es un hipervisor?]

El hipervisor depende del SO anfitrión para acceder a CPU, memoria, red y disco. Hay más capas: syscalls al SO anfitrión, doble scheduler, cambios de contexto adicionales y tiene un overhead mayor que Tipo 1, pero más cómodo para escritorio, pruebas y desarrollo.

Contenedores linux
Aquí no hay hipervisor. El kernel del host ya gestiona el hardware. El contenedor sólo limita y aísla la vista de los procesos.

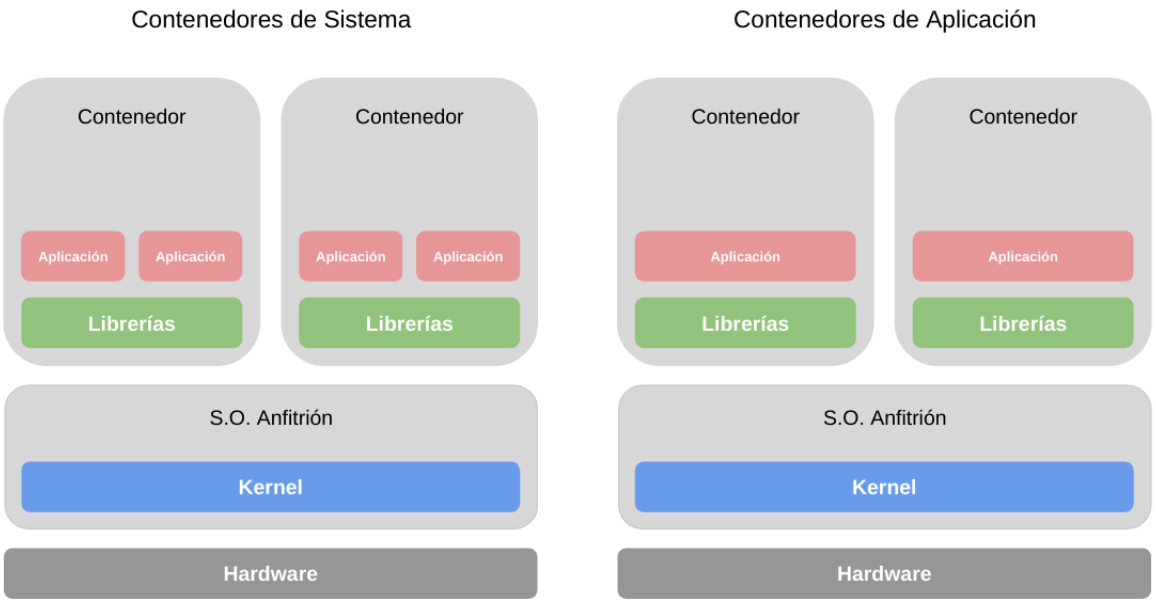


Imagen sacada de la fuente [GNU/Linux. Contenedores – ComputerNewAge]

Donde se destacan los mecanismos principales:

- **Namespaces:** PID, NET, MNT, IPC, UTS, USER.
- **Cgroups v1/v2:** límites de CPU, memoria, I/O, PIDs, etc.
- **Capabilities:** dividen los privilegios de root.
- **Seccomp:** filtra llamadas al sistema.
- **LSM:** AppArmor, SELinux, etc.
- **UnionFS/OverlayFS:** capas de solo lectura + capa de escritura.
- **Rootfs propio:** el contenedor ve su propio sistema de archivos.

Un contenedor no es una VM ligera. Es un grupo de procesos del host con una vista aislada.

¿Por qué el contenedor comparte kernel y la VM necesita kernel invitado?
En una VM, el hipervisor presenta hardware virtual. El Guest OS necesita su propio kernel para ya sea inicializar la CPU virtual, configurar paginación y MMU, manejar interrupciones, cargar drivers, planificar procesos, gestionar memoria y ofrecer la interfaz de syscalls a las aplicaciones. Sin kernel invitado, no hay quien gobierne ese hardware virtual. En un contenedor, no hay hardware virtual. La aplicación hace open(), read(), write(), socket() directamente contra el kernel del host. El aislamiento no viene de virtualizar hardware, sino de limitar qué puede ver y usar ese proceso.

Impacto en CPU, memoria y tiempo de arranque
Sobrecarga de CPU

- Hipervisor Tipo 1: overhead bajo-medio. Hay VM exits, emulación de dispositivos, interrupciones virtuales, EPT/NPT. Con virtio y VT-x/AMD-V se redujo mucho.
- Hipervisor Tipo 2: overhead medio-alto. Además del hipervisor, interviene el SO anfitrión con syscalls, scheduler doble, copias de datos y cambios de contexto.
- Contenedor linux: overhead muy bajo. Las syscalls van directas al kernel. Solo hay coste por cgroups, seccomp, overlayfs, namespaces y planificación normal. No hay VM exits ni kernel invitado.

Consumo de memoria principal

- Hipervisor: cada VM reserva memoria para su kernel, drivers, page cache, servicios y aplicaciones. Una VM Linux mínima puede consumir 512 MB - 1 GB solo de base; una Windows, 2 GB o más. La memoria no se comparte entre VMs, salvo algunos mecanismos como KSM o ballooning.
- Contenedor linux: comparte kernel y page cache del host. El consumo base es de pocos MB. Las capas de imagen se comparten entre contenedores. Si la app usa mucha memoria, igual se limita con memory.max o memory.limit_in_bytes.

Tiempo de arranque

- Hipervisor: firmware -> bootloader -> kernel -> init -> servicios. Suele tardar segundos o minutos.
- Contenedor linux: el kernel ya está cargado. Solo se crean namespaces, cgroups, se monta el rootfs y se ejecuta el proceso. Suele tardar milisegundos o pocos segundos.

Entonces para resumir:

La VM virtualiza hardware y por eso necesita un kernel invitado. Ofrece aislamiento fuerte, permite otro SO y otro kernel, pero paga overhead en CPU, memoria y arranque.

El contenedor no virtualiza hardware, aísla procesos sobre el mismo kernel del host. Es mucho más ligero, denso y rápido, pero no puede ejecutar un kernel distinto ni ofrece el mismo aislamiento que una VM.

Tipo 1 es más eficiente que Tipo 2 porque elimina el SO anfitrión intermediario.

Contenedor es aún más eficiente que ambos porque elimina también el kernel invitado y la virtualización de hardware.

2) Namespaces
Un namespace es una abstracción del kernel que envuelve un recurso global del sistema y hace que los procesos dentro de él vean una instancia propia y aislada de ese recurso. Sin namespaces, todos los procesos del sistema comparten la misma tabla de PIDs, las mismas interfaces de red, los mismos puntos de montaje, las mismas colas IPC,el mismo hostname y los mismos UID/GID. Con namespaces, el kernel puede dar a un grupo de procesos una vista parcial y separada de esos recursos, aunque sigan usando el mismo kernel. A continuación detallo las responsabilidades técnicas de los siguientes namespaces

2.1) PID: Aísla los IDs de proceso y árbol de ejecución. Cada namespace PID tiene su propio contador de PIDs, el primer proceso creado dentro del namespace recibe PID1 dentro de ese namespace, además un proceso tiene dos PIDs, uno en su namespace que es el que ve él, y otro en el namespace padre que es el que ve el host, el kernel mantiene la relación padre hijo dentro del namespace. Los procesos de un namespace PID no pueden ver ni señalar procesos de otros namespace salvo el padre y cuando muere el PID 1 del namespace, el kernel envía SIGKILL a todos los procesos de ese namespace.

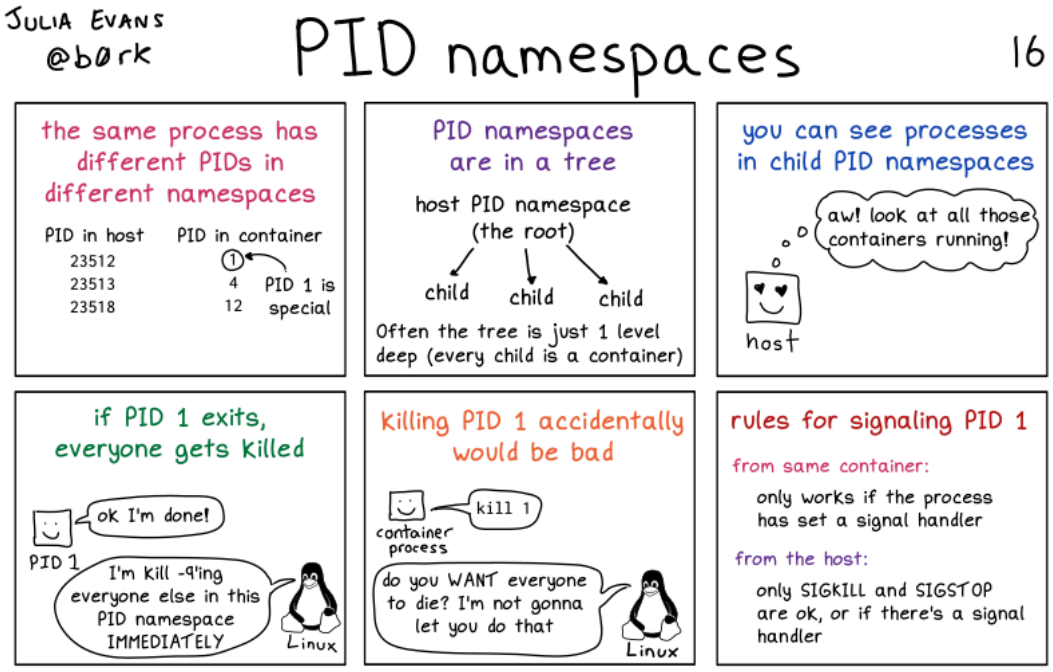


Imagen de la fuente https://wizardzines.com/comics/pid-namespaces/.

2.2) NET: asila la pila de red completa, cada NET tiene su propia lista de interfaces de red, tabla de rutas, tabla ARP/neigh, reglas de implantes/nftables, sockets, puertos, conexiones y nombres de host de red compartido con UTS. Sirven en los contenedores linux, cada contenedor tiene su propia IP, sus puertos y sus reglas de firewall, sin chocar con otros.

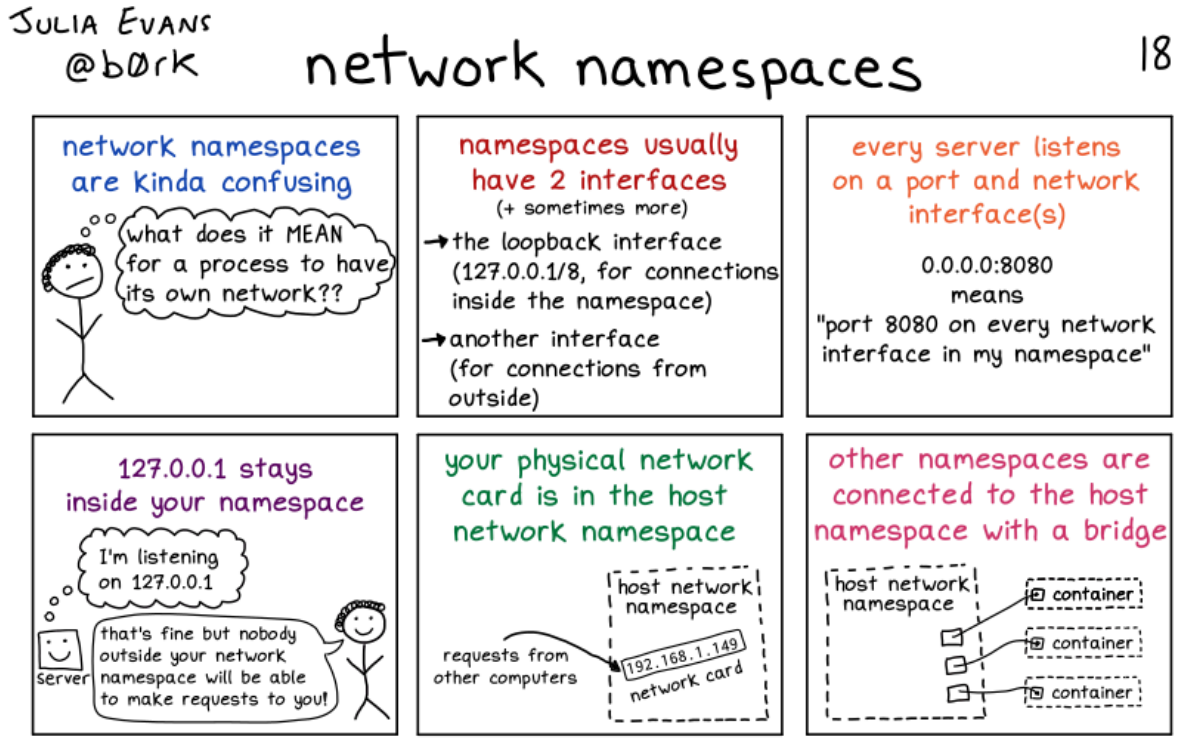


Imagen de la fuente https://wizardzines.com/comics/network-namespaces/.

2.3) MNT: Aísla los puntos de montaje y la jerarquía de sistemas de archivos, cada MNT tiene su tabla de montajes, raíz del sistema de archivos. puntos de montaje vitales, propagación de montajes ya sean shared, slave y private. Estos sirven en los contenedores para dar a cada uno su propio sistema de archivos raíz sin afectar al host.

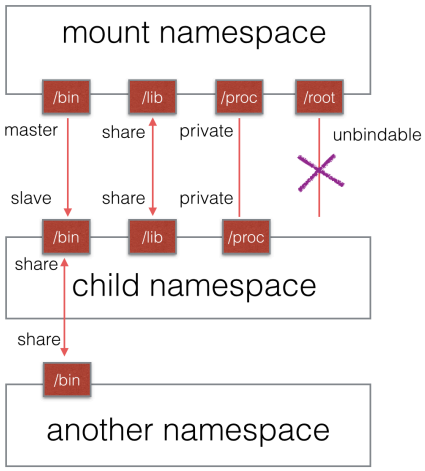


imagen de la fuente Mount namespace | Yin的笔记本.

2.4) IPC: Aísla los mecanismos de comunicación entre procesos (IPC), incluye colas de mensajes y memoria compartida POSIX, colas de mensaje, memoria compartida y semáforos SysV. Cada IPC tiene sus propios identificadores IPC. Dos procesos en namespaces distintos no pueden usar la misma cola o semáforo aunque tengan el mismo ID numérico. Esto evita que un contenedor interfiera con las colas o memoria compartida de otro o del host.

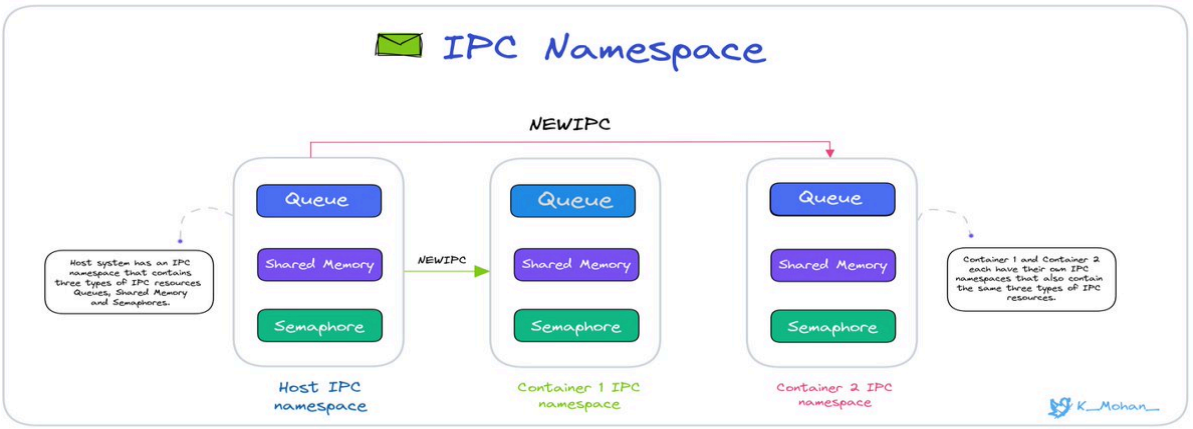


Imagen de la fuente https://threadreaderapp.com/thread/1635484543078789121.

2.5) UTS: Aisla el hostname y el domain name NIS, es un namespace pequeño pero esencial. Permite que cada contenedor tenga su propio nombre del host sin cambiar el del host, y esto ayuda con las apps que dependen del hostname vean un nombre coherente con su rol.

2.6) USER: el mapeo y la traduccion de UID y GID, es el namespace más poderoso para seguridad. Permite que un proceso sea root dentro del contenedor pero no root en el host. Dentro del namespace, los UID/GID se ven como 0, 1, 2, etc. Fuera (en el host), esos mismos UID/GID se traducen a otros valores, normalmente altos (100000+). El kernel traduce cada syscall que usa UID/GID. Esto ayuda a los contenedores a ejecutar rootless, reducir el impacto de una escapada y limitar capacidades.

¿Cómo se configuran estas fronteras?
Hay tres llamadas al sistema principales:

clone(): crea un proceso nuevo en uno o varios namespaces nuevos, tiene las banderas:
CLONE_NEWPID //para PID
CLONE_NEWNET //para NET
CLONE_NEWNS //para MNT
CLONE_NEWIPC // para IPC
CLONE_NEWUTS //para UTS
CLONE_NEWUSER //para USER
CLONE_NEWCGROUP //paraCGROUP

```
clone(child_func, stack,
      CLONE_NEWPID | CLONE_NEWNET | CLONE_NEWNS |
      CLONE_NEWIPC | CLONE_NEWUTS | CLONE_NEWUSER |
      SIGCHLD, NULL);
```

Ejemplo generado con ia para un mejor entendimiento

El child nace ya dentro de los nuevos namespaces

unshare(): Este separa el proceso actual y sus futuros hijos a nuevos namespaces sin crear un proceso nuevo.

```
unshare --pid --fork --mount-proc /bin/bash
```

Ejemplo generado con ia para un mejor entendimiento

Esto crea un nuevo namespacec PID, hace fork para que el bash sea PID1 dentro y monta /proc para que ps fncione correctamente.
Esta frontera es útil cuando ya tenemos un proceso corriendo y queremos aislarlo sin relanzarlo.

setns(): Une un proceso a un namespace ya existente. Se usa, por ejemplo, cuando una herramienta como nsenter quiere entrar al namespace de un contenedor ya en ejecución.

```
nsenter --target <PID> --net --mount --pid
```

Ejemplo generado con ia para un mejor entendimiento

Permitiendo depurar o inspeccionar un contenedor desde dentro sin reiniciarlo.

¿Cómo se pueden auditar en el pseudo-sistema de archivos /proc/<PID>/ns/?
El kernel expone los namespaces de cada proceso como enlaces simbólicos en /proc/<PID>/ns/
Para entenderlo mejor se muestra el siguiente ejemplo

```
ls -l /proc/1234/ns/
lrwxrwxrwx 1 root root 0 ... cgroup -> 'cgroup:[4026531835]'
lrwxrwxrwx 1 root root 0 ... ipc    -> 'ipc:[4026531839]'
lrwxrwxrwx 1 root root 0 ... mnt    -> 'mnt:[4026531840]'
lrwxrwxrwx 1 root root 0 ... net    -> 'net:[4026531992]'
lrwxrwxrwx 1 root root 0 ... pid    -> 'pid:[4026531836]'
lrwxrwxrwx 1 root root 0 ... user   -> 'user:[4026531837]'
lrwxrwxrwx 1 root root 0 ... uts    -> 'uts:[4026531838]'
```

Ejemplo generado con ia para un mejor entendimiento

Cada enlace apunta a un inodo de namespace identificado por un número coomo net:[4026531992]. Si dos procesos apuntan al mismo inodo, comparten ese namespace. si no, están aislados.

3) Cgroups
También conocidos como control groups son una característica del kernel de Linux que permite organizar procesos en grupos jerárquicos y distribuir recursos del sistema de manera controlada. Su rol fundamental abarca cuatro funciones clave mediante las cuales son implementadas mediante controladores o subsistemas que actúan sobre recursos concretos como CPU, memoria, E/S de bloque, red, etc:

1. **Limitar el uso de recursos:** como restringir el porcentaje de CPU, la cantidad de memoria RAM o el ancho de banda de E/S que puede consumir un grupo de procesos.

2. **Priorizar la asignación de recursos:** a través de pesos o shares, se puede indicar qué grupos deben recibir más tiempo de CPU o más ancho de banda de E/S cuando hay contención.

3. **Contabilizar el consumo:** el kernel lleva un registro del uso de recursos tiempo de CPU, memoria, E/S, por cada cgroup, lo que permite auditar y facturar el consumo.

4. **Aislar procesos:** los grupos se mantienen separados de modo que el agotamiento de recursos en uno no afecte a los demás, y se puede impedir el acceso a recursos específicos.

cgroups v2 (jerarquía unificada) vs cgroups v1 (controladores independientes)
Estructura de jerarquía:

- Cgroups v1: Múltiples jerarquías independientes; cada controlador puede montarse en su propia jerarquía.
- Cgroups v2: Una única jerarquía unificada donde todos los controladores disponibles se montan automáticamente.

Flexibilidad:

- Cgroups v1: Permite montar diferentes controladores en diferentes jerarquías, lo que en teoría daba flexibilidad, pero en la práctica añadía complejidad.
- Cgroups v2: Simplifica la administración al tener una sola vista de todos los recursos.

Coordinación entre controladores:

- Cgroups v1: Los controladores pueden pertenecer a jerarquías distintas, lo que dificulta la coordinación (p. ej., CPU y memoria no ven la misma estructura de grupos).
- Cgroups v2: Todos los controladores comparten la misma jerarquía, lo que facilita la coordinación y el control conjunto.

Nombres de archivos:

- Cgroups v1: Los nombres y comportamientos de los archivos de control varían entre controladores.
- Cgroups v2: Los nombres de los archivos son consistentes entre controladores (p. ej., memory.current, cpu.stat).

Procesos internos:

- Cgroups v1: Se permiten procesos en nodos internos de la jerarquía.
- Cgroups v2: Solo se permiten procesos en nodos hoja (excepto el cgroup raíz), lo que simplifica la semántica.

Notificación de grupos vacíos:

- Cgroups v1: Usa el mecanismo release_agent.
- Cgroups v2: Utiliza el archivo cgroup.events (campo populated).

Compatibilidad:

- Cgroups v1: Puede coexistir con v2 en el mismo sistema, pero un controlador no puede usarse simultáneamente en ambos.
- Cgroups v2: Puede montarse junto a v1, pero los controladores no pueden compartirse.

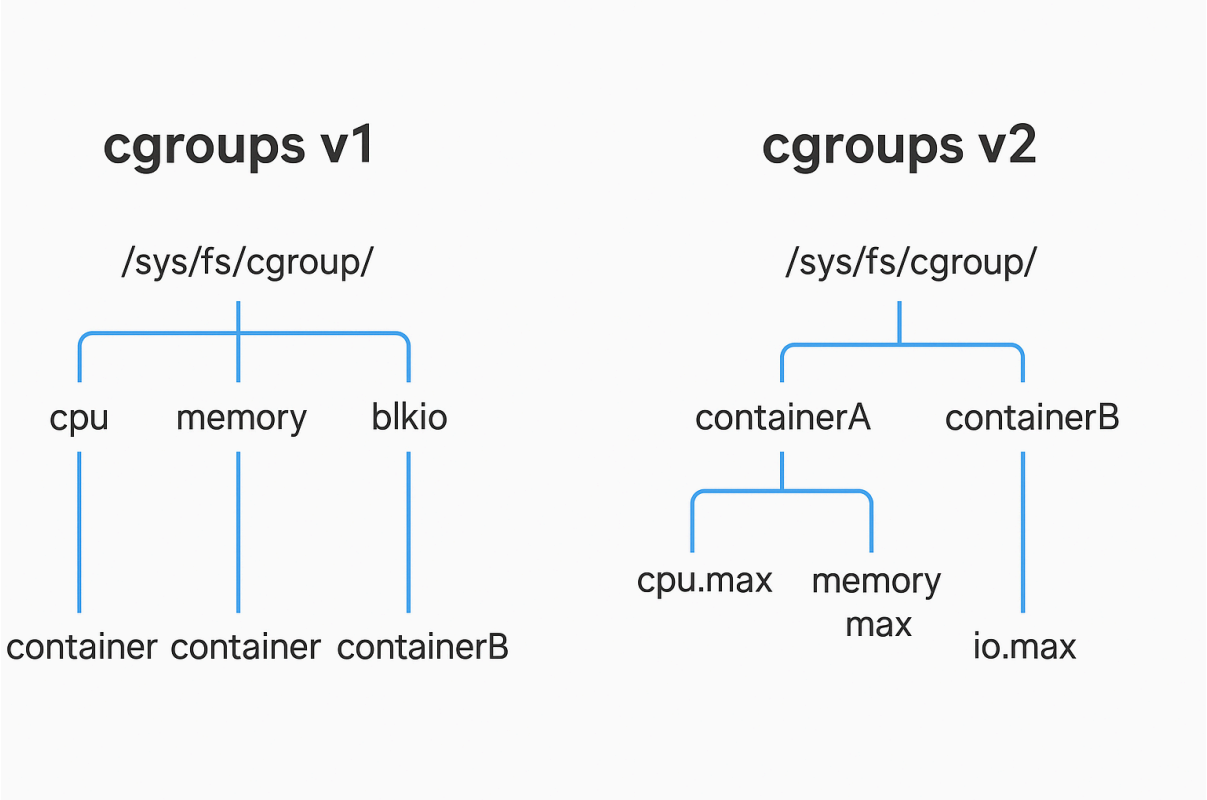


Imagen de la fuente [cgroups v2 — The Unsung Hero Behind Java's Mysterious OutOfMemoryErrors | by Ashish Singh | Medium]

Pseudo-archivos en /sys/fs/cgroup/ que registran consumo
En cgroups v2, dentro del directorio de cada grupo existen archivos de solo lectura que exponen el consumo de recursos:

- Memoria: memory.current. Que muestra la cantidad total de memoria en bytes actualmente en uso por el cgroup y sus descendientes.
  Ejemplo: cat /sys/fs/cgroup/system.slice/docker-<id>.scope/memory.current.

- CPU: cpu.stat. Contiene estadísticas de uso de CPU, como usage_usec, user_usec y system_usec.
  Ejemplo: cat /sys/fs/cgroup/system.slice/docker-<id>.scope/cpu.stat.

En cgroups v1, los archivos equivalentes eran memory.usage_in_bytes (para memoria) y cpuacct.usage (para CPU), pero v2 los unificó en memory.current y cpu.stat para mayor claridad.

4) Completly fair Sheduler (CFS)
En el subsistema de CPU de Linux, el CFS implementa un mecanismo de control de ancho de banda (CFS Bandwidth Control) que permite establecer un límite máximo de tiempo de

CPU para un grupo de procesos. Este mecanismo es la base para la limitación de CPU en contenedores y se apoya en dos parámetros fundamentales: período y cuota.

4.1) Parámetros cpu.cfs_period_us y cpu.cfs_quota_us

cpu.cfs_period_us (anteriormente mencionado como periodo): Define la duración de la ventana temporal en microsegundos durante la cual se evalúa el consumo de CPU de un cgroup. Su valor por defecto es 100.000 µs.

cpu.cfs_quota_us (ya mencionado como cuota): Especifica el tiempo total de CPU de igual manera en microsegundos que todos los procesos del cgroup pueden consumir dentro de cada periodo. Un valor de -1 indica que no hay restricción.

En conjunto, estos parámetros definen el ancho de banda de CPU disponible para el grupo. Si la cuota es menor que el periodo, el grupo está limitado a una fracción de CPU.

4.2) Calculo del limite

Un límite de 0,25 CPUs significa que el contenedor puede usar el equivalente a un cuarto de núcleo. Sobre un período estándar de 100.000 µs, la cuota se calcula así:

Cuota = 0,25 × 100.000 µs = 25.000 µs

Entonces, el contenedor puede consumir como máximo 25.000 microsegundos de tiempo de procesador dentro de cada ventana de 100.000 µs, es decir, 25 ms de CPU por cada 100 ms de tiempo real.

4.3) Comportamiento al agotar la cuota: throttling

Cuando los procesos del cgroup consumen toda la cuota asignada antes de que termine el periodo, el kernel aplica throttling, traducido como estrangulamiento:

- Los procesos del grupo no pueden ejecutarse hasta que comience el siguiente periodo y la cuota se reponga.
- Durante ese intervalo, permanecen en estado throttled, lo que puede provocar picos de latencia y respuestas lentas, incluso si el uso promedio de CPU parece moderado.
- Al inicio del siguiente periodo, la cuota se restablece automáticamente al valor configurado (25.000 µs en el ejemplo).

5) Gestión de memoria en cgroups, el Out-Of-Memory (OOM) Killer y el código de salida 137

Comportamiento del cgroup al exceder la cuota estricta de memoria (memory.max):

En el subsistema de control de recursos de Linux, el archivo memory.max (en cgroups v2) o memory.limit_in_bytes (en cgroups v1) define el límite duro de memoria física que un grupo de procesos puede consumir. A diferencia de memory.high, que actúa como un umbral de estrangulamiento (throttling) y presión de recuperación, memory.max es el mecanismo principal para imponer un techo de memoria efectivo.

Cuando el uso de memoria de un cgroup alcanza este límite, el kernel activa de inmediato un proceso de reclamación (memory reclaim). Este mecanismo intenta liberar memoria

reciclando páginas de caché, buffers y otras estructuras no esenciales. Si la reclamación logra reducir el consumo por debajo del límite, el sistema continúa operando con normalidad. Sin embargo, si la presión de memoria persiste y no es posible recuperar suficientes páginas para satisfacer las nuevas solicitudes de asignación, el kernel invoca al Out-Of-Memory Killer.

Por Ejemplo: Si se configura un contenedor con memory.max = 256 MB y una aplicación dentro de él intenta asignar 300 MB, el kernel primero intentará liberar memoria caché. Si no lo consigue, el OOM Killer se activará y seleccionará uno o varios procesos del cgroup para ser terminados, de modo que el consumo vuelva a estar por debajo de los 256 MB.

Condiciones de intervención del OOM Killer y la señal SIGKILL: El OOM Killer es un mecanismo de último recurso del kernel de Linux. Su intervención se activa bajo condiciones específicas las cuales son:

1. **Memoria global insuficiente:** El sistema en su conjunto no puede satisfacer una solicitud de asignación de memoria, incluso después de intentar reclamar páginas de todos los cgroups.
2. **Cuota de cgroup excedida:** El uso de memoria de un cgroup alcanza su límite duro (memory.max o memory.limit_in_bytes) y la reclamación no logra reducirlo.
3. **Memoria de un cgroup padre insuficiente:** Un cgroup hijo excede la cuota de su cgroup padre, aunque el hijo no haya alcanzado su propio límite.
4. **Asignación de memoria no recuperable:** El kernel necesita asignar memoria de forma urgente, por ejemplo, para operaciones de E/S críticas y no hay páginas disponibles.

Cuando se cumplen estas condiciones, el OOM Killer selecciona un proceso víctima basándose en una puntuación de mala conducta oom_score, que pondera factores como el consumo de memoria, la prioridad del proceso y si es un proceso del sistema. Una vez seleccionado, el kernel le envía la señal SIGKILL o señal número 9, que no puede ser capturada, bloqueada ni ignorada por el proceso. La SIGKILL provoca la terminación inmediata del proceso víctima, liberando la memoria que ocupaba.

Justifique matemáticamente por qué los contenedores finalizados por este mecanismo reportan al sistema el código de salida 137: Cuando el OOM Killer termina un contenedor, el sistema reporta un código de salida 137. Este valor no es arbitrario; sigue una convención establecida por POSIX y adoptada por los shells de UNIX:

Código de salida = 128 + N

Donde N es el número de la señal que provocó la terminación. En el caso del OOM Killer, la señal es SIGKILL, cuyo número es 9. Por lo tanto:

Código de salida = 128 + 9 = 137

Esta fórmula proviene de la forma en que el shell (bash, sh, etc.) interpreta la terminación de un proceso hijo. Cuando un proceso es terminado por una señal, el shell retorna 128 + N, donde N es el número de la señal. Esto permite distinguir entre una terminación normal (código 0) y una terminación anómala por señal (código > 128).

Ejemplo completo: Suponiendo que creamos un contenedor con un límite de 128 MB y una aplicación que intenta consumir 200 MB ocurre:

```
docker run --memory=128m --name test-oom alpine sh -c "dd if=/dev/zero of=/dev/null bs=1M count=200"
```

Ejemplo generado con ia para un mejor entendimiento

1. El cgroup del contenedor tiene memory.max = 134217728 bytes (128 MB).
2. La aplicación comienza a asignar memoria y supera los 128 MB.
3. El kernel intenta reclamar memoria, pero no puede liberar suficiente.
4. El OOM Killer se activa, selecciona el proceso dd y le envía SIGKILL.
5. El contenedor termina con código 137.
6. docker inspect test-oom muestra "OOMKilled": true en la sección State.

.

6) Sistemas de archivos de unión (UnionFS), Copia en Escritura (CoW) y el almacenamiento compartido de capas

Funcionamiento de un sistema de archivos de unión (UnionFS).

Un sistema de archivos de unión es una tecnología que nos permite superponer múltiples sistemas de archivos denominados capas o layer en un único punto de montaje, de modo que el resultado se presenta al usuario como un árbol de directorios unificado. A diferencia de un montaje tradicional, que oculta el contenido del directorio subyacente, un montaje de unión fusiona el contenido de todas las capas, mostrando una vista combinada de todas ellas.

Con respecto al contexto de Docker, el driver de almacenamiento más utilizado es OverlayFS, que es una implementación de UnionFS integrada en el kernel de Linux. OverlayFS opera a nivel de dentry e inodo, multiplexando dos o más árboles de directorios en un espacio de nombres unificado que se presenta al espacio de usuario.

Ejemplo: Suponiendo que tenemos dos directorios:
/capa_base que contiene el archivo app.py.
/capa_superior contiene el archivo config.txt.

Al montar un UnionFS con estas dos capas, el punto de montaje /merged mostrará ambos archivos: app.py y config.txt, como si estuvieran en el mismo directorio, aunque físicamente residan en ubicaciones separadas.

```
vboxuser@ubuntu:~/Desktop/prelab7/ejemplo-unionfs$ ls -la merged/
total 12
drwxrwxr-x 1 vboxuser vboxuser 4096 Oct 12 02:52 .
drwxrwxr-x 7 vboxuser vboxuser 4096 Oct 12 02:52 ..
-rw-rw-r-- 1 vboxuser vboxuser   27 Oct 12 02:52 app.py
vboxuser@ubuntu:~/Desktop/prelab7/ejemplo-unionfs$
```

Ejemplo generado por ia para un mejor entendimiento. ***Ejemplos 6.1***

Principio de Copia en Escritura (Copy-on-Write, CoW).

El principio de CoW es una técnica de optimización que retrasa la copia de datos hasta que es estrictamente necesario modificarlos. En lugar de duplicar inmediatamente un recurso ya sea una página de memoria o un archivo en disco, el sistema permite que múltiples entidades compartan la misma copia de solo lectura. Solo cuando una de ellas intenta

escribir sobre ese recurso, el sistema realiza una copia privada de la porción modificada, dejando intacta la copia original para las demás entidades.

Este mecanismo es fundamental en la contenerización, ya que permite que múltiples contenedores compartan las capas de solo lectura de una misma imagen sin duplicar el espacio en disco. Cuando un contenedor necesita modificar un archivo que reside en una capa inferior, el driver de almacenamiento realiza una operación de copy-up: copia el archivo completo a la capa superior, que es la capa de escritura del contenedor y redirige las operaciones de escritura hacia esa copia, mientras que el archivo original permanece inmutable en la capa inferior.

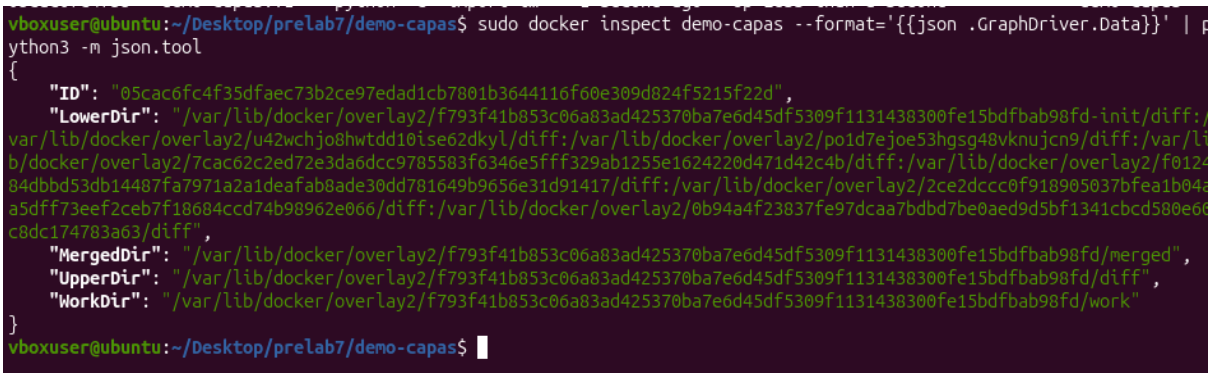
Ejemplo: Entonces, podemos verlo de la siguiente manera. Si dos contenedores se instancian desde la misma imagen python:3.11-alpine por ejemplo, ambos comparten la misma capa de solo lectura que contiene el intérprete de Python. Si el contenedor A modifica /etc/hosts, OverlayFS copia ese archivo a la capa de escritura del contenedor A. El contenedor B sigue viendo el /etc/hosts original de la imagen. El espacio en disco solo se incrementa por el archivo copiado, no por toda la imagen.

```
vboxuser@ubuntu:~/Desktop/prelab7/ejemplo-unionfs$ sudo docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED          STATUS          PORTS
                  NAMES
552d242d90b4   python:3.11-alpine   "sleep 3600"             10 seconds ago   Up 9 seconds
                  contenedor-B
34d172b57cbf   python:3.11-alpine   "sleep 3600"             11 seconds ago   Up 10 seconds
                  contenedor-A
b7fe1935ee0b   lab-so-web:v1        "python app.py"          6 hours ago      Up 2 hours      0.0.0.0:8080->5000/tcp,
[::]:8080->5000/tcp   nodo_web
58bd5e1cb5f4   redis:alpine         "docker-entrypoint.s…"   6 hours ago      Up 2 hours      6379/tcp
                  nodo_db
```

Ejemplo generado por ia para un mejor entendimiento. ***Ejemplos 6.2***

Definición de las capas: lowerdir, upperdir y merged.
En la terminología de OverlayFS, existen capas que componen el sistema de archivos unificado, las cuáles son

- **lowerdir**. Capa inferior, de solo lectura: Capa base que contiene la imagen de solo lectura del contenedor. Puede estar compuesta por múltiples capas. Todos los contenedores que usan la misma imagen comparten estas capas lowerdir, lo que reduce drásticamente el uso de disco. En Docker, las capas de la imagen se montan como una lista de directorios separados por dos puntos en la opción lowerdir.
- **upperdir**. Capa superior, de lectura/escritura: Es la capa de trabajo específica de cada contenedor. Aquí se almacenan todas las modificaciones realizadas por el contenedor: archivos nuevos, archivos modificados y archivos eliminados representados mediante whiteouts. Es la única capa que se escribe durante la ejecución del contenedor.
- **merged.** Punto de montaje consolidado: Es el directorio virtual que se presenta al contenedor como su sistema de archivos raíz. No almacena datos por sí mismo; es el resultado de la fusión de lowerdir y upperdir mediante el montaje OverlayFS. Cuando un proceso dentro del contenedor abre un archivo, el kernel resuelve la ruta buscando primero en upperdir y, si no lo encuentra, en lowerdir (de arriba hacia abajo). Al listar un directorio, se combinan las entradas de todas las capas, aplicando las reglas de precedencia, la capa superior oculta a la inferior en caso de conflicto.



Ejemplo generado por ia para un mejor entendimiento. ***Ejemplos 6.3***

¿Por qué dos contenedores de la misma imagen no duplican el espacio base?.
La principal razón es que las capas de la imagen lowerdir son de solo lectura como se mencionó anteriormente y se comparten entre todos los contenedores que derivan de ella. Cuando Docker crea un contenedor, no copia la imagen completa; simplemente crea una nueva capa de escritura (upperdir) vacía y monta el sistema de archivos unificado apuntando a las capas de la imagen como lowerdir y a la nueva capa como upperdir.

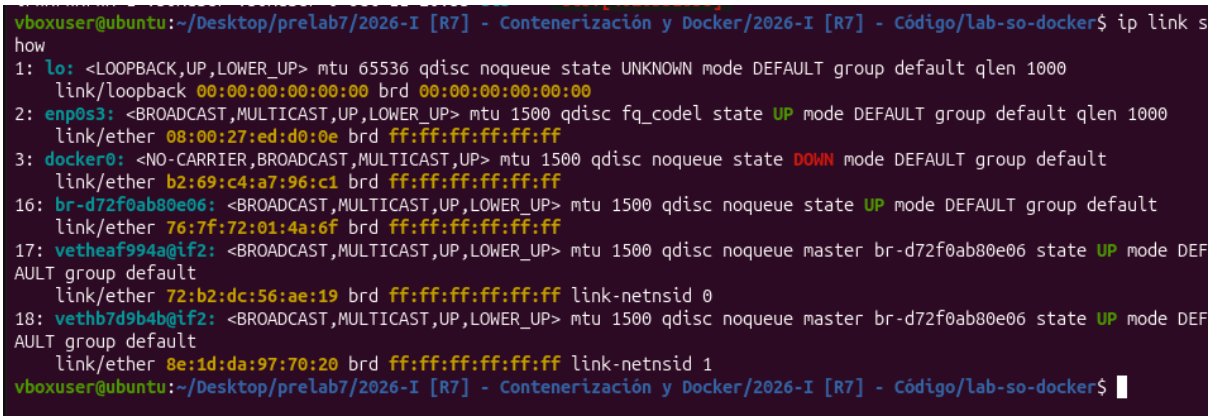
7) Redes virtuales en Docker: veth pairs, Linux Bridge y NAT con iptables

Cada contenedor Docker se ejecuta dentro de su propio network namespace, una de las abstracciones fundamentales del kernel de Linux para el aislamiento de recursos. Un network namespace proporciona una pila de red completamente independiente: sus propias interfaces, tablas de enrutamiento, reglas de firewall iptables, sockets y puertos. Esto significa que un proceso dentro del contenedor sólo puede ver las interfaces de red que residen en su namespace; no tiene visibilidad de las interfaces del host ni de las de otros contenedores.
Un **veth** pair es un dispositivo de red virtual de Capa 2 que actúa como un cable de Ethernet virtual. Se compone de dos interfaces que están conectadas entre sí: todo lo que entra por una sale por la otra, y viceversa. La característica clave es que cada extremo del par puede residir en un namespace de red diferente, lo que permite conectar la pila de red de un contenedor con la del host. Cuando Docker inicia un contenedor, el daemon realiza las siguientes operaciones atómicas:

- Crea un veth pair con dos interfaces, por ejemplo vethA en el host y vethB para el contenedor.
- Mueve una de las interfaces al network namespace del contenedor.
- Dentro del contenedor, esa interfaz se renombra a eth0, convirtiéndose en la interfaz principal del contenedor.
- La otra interfaz, vethA permanece en el namespace del host y se conecta al bridge virtual.

Podemos ver este par de interfaces en nuestro host con el comando ip link show, donde aparecerán entradas como vethXXXX@ifN, donde @ifN indica el índice de la interfaz en el

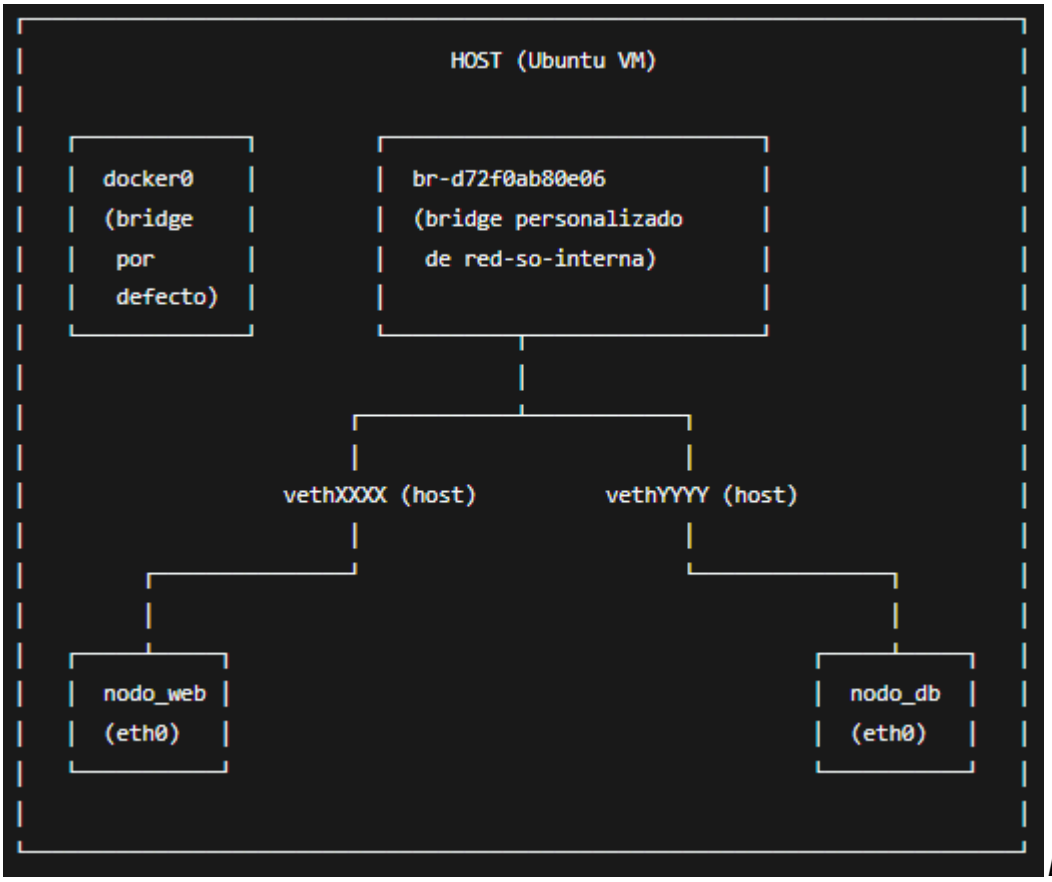
otro extremo del cable virtual dentro del contenedor. Algo que veremos en el prelaboratorio



Linux Bridge es un dispositivo de red virtual que implementa un conmutador switch de Capa 2 por software. Funciona de manera análoga a un switch físico: reenvía tramas Ethernet entre las interfaces que tiene conectadas, aprendiendo las direcciones MAC de cada dispositivo para dirigir el tráfico de forma eficiente.

Docker utiliza bridges para interconectar los veth pairs de los contenedores:

- docker0: Bridge por defecto que Docker crea al iniciarse. Todos los contenedores que no especifican una red personalizada se conectan a él.
- Bridges personalizados (br-XXXX): Cuando usas docker-compose con una red definida por el usuario como red-so-interna en tu laboratorio, Docker crea un bridge dedicado con un nombre aleatorio (ej. br-d72f0ab80e06). Este bridge aísla el tráfico de esa red del resto de contenedores y del host.



Ejemplo generado con ia para un mejor entendimiento

En este ejemplo los contenedores nodo_web y nodo_db están conectados al bridge br-d72f0ab80e06 mediante sus respectivos veth pairs. El bridge actúa como un switch que permite la comunicación entre ambos contenedores, pero los aísla del resto del sistema.

Ahora, **¿Cómo se realiza la traslación de direcciones de red (NAT / iptables) en el host cuando se expone un puerto hacia el exterior mediante la bandera-p 8080:5000?**
Cuando ejecutamos un contenedor con la bandera -p 8080:5000, Docker publica el puerto 5000 del contenedor en el puerto 8080 del host. Para lograrlo, el daemon de Docker inserta reglas en la tabla nat de iptables que realizan DNAT (Destination NAT) y SNAT/MASQUERADE.

- Los contenedores tienen direcciones IP privadas dentro de una red bridge (como 172.18.0.2). Estas direcciones no son enrutables desde fuera del host. Si un cliente externo intenta conectarse a 172.18.0.2:5000, el paquete no encontrará ruta. Se necesita traducir las peticiones dirigidas al puerto 8080 del host hacia el puerto 5000 del contenedor.

Al publicar un puerto, Docker añade reglas en la tabla nat de iptables. Las más importantes son:

- Cadena PREROUTING: se evalúa antes de que el paquete llegue a la pila de red local. Docker añade una regla que salta a la cadena DOCKER para paquetes cuyo destino es una dirección local del host.
- Cadena OUTPUT: se evalúa para paquetes generados localmente (por ejemplo, desde localhost). Docker añade una regla similar para tráfico local.
- Cadena DOCKER: aquí está la regla DNAT específica:

```
iptables -t nat -A DOCKER -p tcp --dport 8080 -j DNAT --to-destination 172.18.0.2:5000
```

Ejemplo generado con ia para un mejor entendimiento
Esta regla dice: Todo paquete TCP que llegue con puerto destino 8080, reescribe su destino a 172.18.0.2:5000.

- Cadena POSTROUTING: Docker añade una regla MASQUERADE para el tráfico saliente de los contenedores, que reescribe la dirección de origen para que las respuestas vuelvan correctamente.

8) Riesgos operativos de ejecutar la aplicación como PID 1 sin un manejador de señales
El rol tradicional de init en UNIX.
En un sistema UNIX convencional, el proceso init o systemd en distribuciones modernas es el primer proceso que se ejecuta tras el arranque del kernel, y recibe el PID 1. Sus responsabilidades son dos:

1. Adoptar procesos huérfanos: Cuando un proceso padre termina antes que sus hijos, esos hijos quedan huérfanos y son re-adoptados por init, que se convierte en su nuevo padre.
2. Reapear procesos zombies: Cuando un proceso hijo termina, su entrada en la tabla de procesos permanece hasta que el padre llama a wait() o waitpid() para leer su estado de salida. init realiza esta operación de forma automática, evitando la acumulación de zombies.

El comportamiento especial del kernel con PID 1.
El kernel de Linux trata al proceso con PID 1 de forma especial en lo que respecta al manejo de señales. Mientras que un proceso normal, al recibir una señal para la que no se ha registrado un manejador, ejecuta la acción por defecto (ya sea terminar, ignorar, etc.), el PID 1 no ejecuta las acciones por defecto para las señales. Solo actúa si el proceso ha registrado explícitamente un manejador para esa señal. Las únicas excepciones son SIGKILL y SIGSTOP, que no pueden ser capturadas ni ignoradas y siempre se aplican.

El problema en contenedores: la aplicación como PID 1.
Cuando Docker inicia un contenedor, el primer proceso que se ejecuta definido por ENTRYPOINT o CMD recibe el PID 1 dentro del PID namespace del contenedor. Si ese proceso es la aplicación directamente (python app.py, node server.js, dotnet MyApp.dll), la aplicación hereda las responsabilidades de init sin haber sido diseñada para ello.

Riesgo 1: El SIGTERM es ignorado
El flujo de detención de un contenedor es el siguiente:

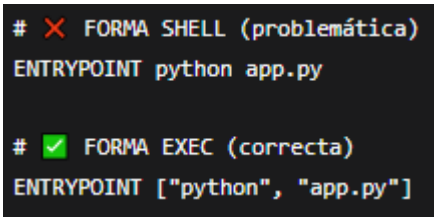
- docker stop envía SIGTERM al PID 1 del contenedor.
- Docker espera un periodo de gracia que por defecto, son 10 segundos.
- Si el PID 1 no ha terminado, Docker envía SIGKILL, que fuerza la terminación inmediata.

Si la aplicación no ha registrado un manejador para SIGTERM, el kernel descarta la señal para PID 1. La aplicación nunca se entera de que debe detenerse, y el contenedor espera los 10 segundos completos antes de ser terminado forzosamente con SIGKILL.
Esto genera las consecuencias de pérdida de datos, conexiones activas cortadas y recursos no librados.

Riesgo 2: Acumulación de procesos zombie
Si la aplicación PID 1 crea procesos hijos (mediante subprocess.Popen, child_process.spawn, Process.Start por ejemplo), esos hijos quedan huérfanos cuando terminan, porque la aplicación no llama a wait(). El kernel re-adopta a esos huérfanos bajo PID 1, pero como PID 1 no realiza el reaping, los procesos terminados permanecen como zombies en la tabla de procesos.

Un proceso zombie consume muy poca memoria, solo su descriptor de proceso, pero ocupa una entrada en la tabla de PIDs. Los sistemas UNIX tienen un número finito de PIDs que por defecto es 32768 en Linux. Si los zombies se acumulan rápidamente, pueden agotar el pool de PIDs, impidiendo la creación de nuevos procesos. Y cuando esto ocurre, fork() falla con el error EAGAIN (Resource temporarily unavailable), y la aplicación no puede seguir funcionando.

Riesgo 3: El script de shell como PID 1.
Un caso particularmente problemático es cuando el ENTRYPOINT usa la forma shell en lugar de la forma exec:



Ejemplo generado con ia para un mejor entendimiento

En la forma shell, Docker ejecuta /bin/sh -c "python app.py". El shell se convierte en PID 1, y python app.py se ejecuta como su hijo PID 2. El SIGTERM de docker stop llega al shell, pero el shell no lo reenvía a su hijo. La aplicación nunca recibe la señal y el contenedor se termina por timeout con SIGKILL.

Prelaboratorio

1. **Elaboración del Archivo de Construcción (Dockerfile)**

Este archivo describe las instrucciones para la creación de la imagen del servicio web partiendo de un sistema base mínimo (alpine) e instalando únicamente las capas estrictamente necesarias:

```
  GNU nano 8.7.1                                        Dockerfile
# 1. Capa Base: Distribucion ligera de Linux con entorno de ejecucion
FROM python:3.11-alpine

# 2. Variables de entorno del sistema operativo dentro del contenedor
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1

# 3. Directorio de trabajo en el sistema de archivos del contenedor
WORKDIR /app

# 4. Aprovechamiento del Layer Caching: Copiar dependencias e instalarlas
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 5. Copia del codigo fuente hacia el sistema de archivos de la capa superior
COPY app.py .

# 6. Metadata de documentacion: Puerto que el proceso escuchara
EXPOSE 5000

# 7. Punto de entrada (PID 1): Ejecucion del servidor
CMD ["python", "app.py"]
```

Aquí investigue qué hace cada uno de estos puntos:

- FROM python:3.11-alpine -> Usa una base mínima con Python 3.11. Alpine reduce tamaño.
- ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 -> Evita archivos .pyc y fuerza logs sin buffer.
- WORKDIR /app -> Define /app como directorio de trabajo dentro del contenedor.
- COPY requirements.txt . -> Copia primero dependencias para aprovechar caché de capas.
- RUN pip install --no-cache-dir -r requirements.txt -> Instala Flask y Redis sin caché.
- COPY app.py . -> Copia el código fuente.
- EXPOSE 5000 -> Documenta que la app escucha en el puerto 5000.
- CMD ["python", "app.py"] -> Ejecuta la app como proceso principal, PID 1.

Cuando procedí con el comando docker build tuve unos problemas, no me reconoció el comando docker build -t lab-so-web:v1. Por lo que tuve que utilizar un poco de ia para descargar docker correctamente y tampoco tenía unos comandos que utiliza el docker por lo que aquí hago presente los comandos que utilice:

Primero Instale curl y herramientas gpg

```
sudo apt update
sudo apt install -y curl gnupg ca-certificates lsb-release
```

Luego descargue la clave de docker y la registre en el sistema con el comando

```
# Crear el directorio para las claves (por si no existe)
sudo install -m 0755 -d /etc/apt/keyrings

# Descargar la clave GPG de Docker directamente a un archivo
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /
etc/apt/keyrings/docker.gpg

# Dar permisos de lectura al archivo de la clave
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

Luego añadi el repositorio oficial de docker

```
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
\
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Actualice el apt y instale el docker, con esto se podra encontrar los paquetes

```
# Actualizar la lista de paquetes (ahora el repositorio de Docker será leído sin
errores)
sudo apt update

# Instalar Docker Engine y los plugins
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin do
cker-compose-plugin
```

y verifique que tenga la versión correcta. la siguiente captura es de mi terminal

```
vboxuser@ubuntu:~/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker$ docker --
version
docker compose version
Docker version 29.9.0, build f415da8
Docker Compose version v5.6.0
```

Ahora si con esto podemos continuar en donde nos quedamos, ejecutando el comando docker build -t lab-so-web:v1.

```
vboxuser@ubuntu:~/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker$ sudo dock
er build -t lab-so-web:v1 .
[+] Building 27.7s (10/10) FINISHED                                                                    docker:default
 => [internal] load build definition from Dockerfile                                                           0.1s
 => => transferring dockerfile: 735B                                                                           0.0s
 => [internal] load metadata for docker.io/library/python:3.11-alpine                                          1.4s
 => [internal] load .dockerignore                                                                              0.1s
 => => transferring context: 35B                                                                               0.0s
 => [1/5] FROM docker.io/library/python:3.11-alpine@sha256:d9368b3a5ac59afea7b5d4f2e2aea0941dbf9fdee9c369c5bec0  4.5s
 => => resolve docker.io/library/python:3.11-alpine@sha256:d9368b3a5ac59afea7b5d4f2e2aea0941dbf9fdee9c369c5bec0  0.1s
 => => sha256:82ceee3231d4eac86c0ddc609b706fd9155e8cf76395fcfa9fc419a3eeb7c45c 249B / 249B                      0.1s
 => => sha256:f1244e1e87fef06de2b01420033ab5ad634d1ee6e53c94f0311a3608786494cf 18.72MB / 18.72MB                3.4s
 => => sha256:2c95de37c4a66e94a2fd637591d5c573eb939469cf19c8eb0615051cd9219617 416.29kB / 416.29kB              0.7s
 => => sha256:e2de96513ba9eb53b431787ec8a65cdde380ac4772a3e4c4b714dcfde2a102b5 3.85MB / 3.85MB                  1.4s
 => => extracting sha256:e2de96513ba9eb53b431787ec8a65cdde380ac4772a3e4c4b714dcfde2a102b5                       0.6s
 => => extracting sha256:2c95de37c4a66e94a2fd637591d5c573eb939469cf19c8eb0615051cd9219617                       0.5s
 => => extracting sha256:f1244e1e87fef06de2b01420033ab5ad634d1ee6e53c94f0311a3608786494cf                       0.9s
 => => extracting sha256:82ceee3231d4eac86c0ddc609b706fd9155e8cf76395fcfa9fc419a3eeb7c45c                       0.0s
 => [internal] load build context                                                                              0.1s
 => => transferring context: 1.08kB                                                                            0.0s
 => [2/5] WORKDIR /app                                                                                         5.9s
 => [3/5] COPY requirements.txt .                                                                              0.1s
 => [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                  10.1s
 => [5/5] COPY app.py .                                                                                        0.7s
 => exporting to image                                                                                         4.1s
 => => exporting layers                                                                                        2.1s
 => => exporting manifest sha256:a9fa7d7fc6b541827c5df25a49881129bd34b605b86cc65d2742fe78bd8f4127              0.0s
 => => exporting config sha256:4795f8f1fb66efe1a53fe48b4390e342a72cc4ded371fc13bd59ed1cd536848b                0.0s
 => => exporting attestation manifest sha256:a30099a090b29a4177dbad682d5816fbc228372c861c0141151b2a58b7f9e3a1  0.0s
 => => exporting manifest list sha256:935ccc42176ded03b5fd610b51d56f2988da6c6d639a88f4d3d61ec92087efe6         0.0s
 => => naming to docker.io/library/lab-so-web:v1                                                               0.0s
 => => unpacking to docker.io/library/lab-so-web:v1                                                            1.5s
vboxuser@ubuntu:~/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker$
```

como pueden ver utilize el codigo sudo docker build -t lab-so-web:v1 en vez del docker build -t lab-so-web:v1, esto debido a que sin poner el sudo me salia el siguiente error:

```
vboxuser@ubuntu:~/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker$ docker bu
ild -t lab-so-web:v1 .
ERROR: permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
```

Investigue y esto significa que el socket de Docker (/var/run/docker.sock) pertenece al usuario root y al grupo docker, y mi usuario vboxuser todavía no pertenecía a ese grupo, por lo que la solución fue utilizar sudo al principio. Sudo significa "superuser do" o "substitute user do". Investigue y resumiendo, básicamente es un programa de Linux que permite ejecutar un comando con los privilegios de otro usuario, por defecto el administrador del sistema.

Ahora seguimos, "Inspeccione las capas generadas en el almacenamiento local ejecutando docker images":

```
vboxuser@ubuntu:~/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker$ sudo dock
er images
                                                                                    i Info →  U  In Use
IMAGE           ID             DISK USAGE   CONTENT SIZE   EXTRA
lab-so-web:v1   935ccc42176d       104MB         25.4MB
vboxuser@ubuntu:~/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker$
```

IMAGE: lab-so-web, es nuestra imagen personalizada. ID es el identificador único de la imagen. DISK USAGE es el tamaño total de la imagen. CONTENT SIZE es el tamaño del contenido único de esta imagen, excluyendo las capas que comparte con otras imágenes.

2. **Orquestación Multicontenedor (docker-compose.yml)**

Aquí interconectamos el servicio web y la base de datos

```
 GNU nano 8.7.1                                  docker-compose.yml *
version: '3.8'

services:
  web:
    build: .
    image: lab-so-web:v1
    container_name: nodo_web
    restart: unless-stopped
    ports:
      - "8080:5000"
    environment:
      - REDIS_HOST=nodo_db
      - REDIS_PORT=6379
    networks:
      - red-so-interna
    depends_on:
      - db-redis

  db-redis:
    image: redis:alpine
    container_name: nodo_db
    restart: unless-stopped
    command: redis-server --appendonly yes
    volumes:
      - datos-redis:/data
    networks:
      - red-so-interna

networks:
  red-so-interna:
    driver: bridge

volumes:
```

```
volumes:
  datos-redis:
    driver: local
```

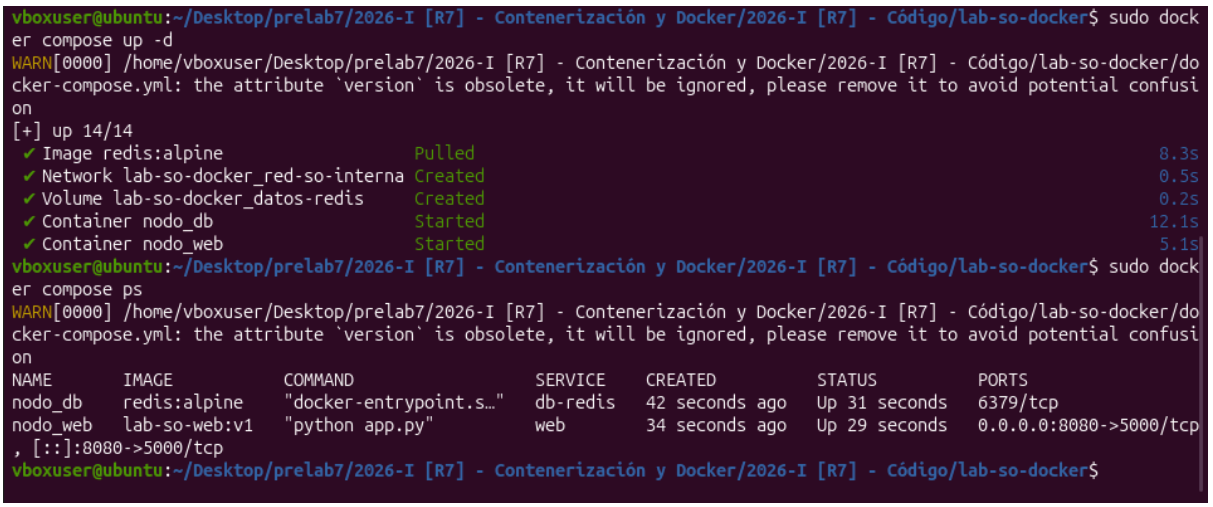
En donde investigue el significado de algunas líneas clave:

- build: . Le dice a Compose que construya la imagen usando el Dockerfile de la carpeta actual.
- image: lab-so-web:v1: Le asigna el nombre y tag a la imagen resultante.
- container_name: nodo_web y nodo_db: Nombres fijos para los contenedores.
- ports: - "8080:5000": Mapea el puerto 8080 de nuestra máquina anfitrión al puerto 5000 del contenedor. Así entramos por http://127.0.0.1:8080/.
- environment: Inyecta las variables de entorno REDIS_HOST y REDIS_PORT que lee nuestro código Python.
- depends_on: - db-redis: Ejecuta primero la base de datos y luego el servicio web.

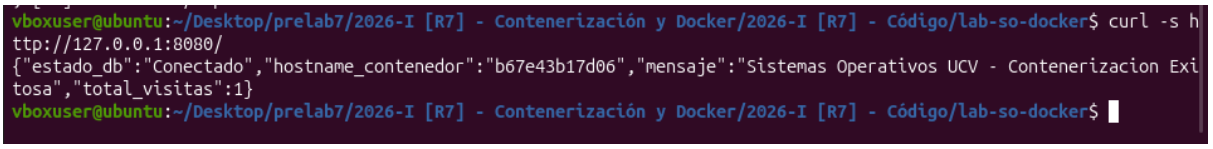
- command: redis-server --appendonly yes: Ejecuta Redis con persistencia AOF activada.
- volumes: - datos-redis:/data: Monta el volumen datos-redis en /data del contenedor, donde Redis guarda sus datos.
- networks: - red-so-interna: Conecta ambos contenedores a una red bridge privada para que se comuniquen por DNS interno.

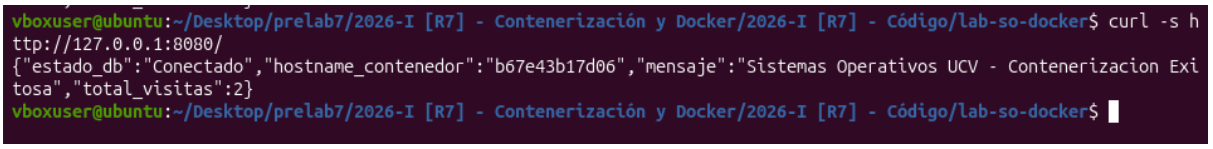
3. Ciclo de Vida, Persistencia y Validación Práctica

Ahora ejecutamos una secuencia de validación, siguiendo las instrucciones:
3.1. Inicie ambos contenedores en segundo plano (detached mode) con docker compose up -d. y verifique el estado operativo de los procesos mediante: docker compose ps, (puse sudo al principio de ellos porque todavía no me agarraban).



3.2. Luego hice múltiples consultas HTTP hacia el puerto publicado del host para comprobar que la co-municación interna por DNS entre nodo_web y nodo_db funciona correctamente mediante el comando curl -s http://127.0.0.1:8080/.





total visitas incrementa, todo bien de momento.
3.3. Ahora hay que demostrar que los datos sobreviven a la destrucción del ciclo de vida del contenedor, para ello detenemos y eliminamos los contenedores en ejecución con el comando docker compose down. Comprobé que el volumen administrado sigue intacto en el host listando los recursos con docker volume ls como decía las instrucciones.

```
vboxuser@ubuntu:~/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker$ sudo dock
er compose down
WARN[0000] /home/vboxuser/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker/do
cker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusi
on
[+] down 3/3
 ✔ Container nodo_web                  Removed                                                                  11.6s
 ✔ Container nodo_db                   Removed                                                                   1.3s
 ✔ Network lab-so-docker_red-so-interna Removed                                                                  0.3s
vboxuser@ubuntu:~/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker$ sudo dock
er volume ls
DRIVER    VOLUME NAME
local     lab-so-docker_datos-redis
```

Luego volví a levantar el entorno, y hice una petición, como pueden ver no se reinicio el total visitas.

```
vboxuser@ubuntu:~/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker$ sudo dock
er compose up -d
WARN[0000] /home/vboxuser/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker/do
cker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusi
on
[+] up 3/3
 ✔ Network lab-so-docker_red-so-interna Created                                                                  0.1s
 ✔ Container nodo_db                   Started                                                                   2.3s
 ✔ Container nodo_web                  Started                                                                   1.5s
vboxuser@ubuntu:~/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker$ curl -s h
ttp://127.0.0.1:8080/
{"estado_db":"Conectado","hostname_contenedor":"1ad4d17f6277","mensaje":"Sistemas Operativos UCV - Contenerizacion Exi
tosa","total_visitas":3}
vboxuser@ubuntu:~/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker$
```

3.4. Ahora Comprobé la diferencia entre remover únicamente los contenedores frente a destruir el almace-namiento persistente con la bandera -v. Ahora si remuevo la bandera -v, que elimina todos los volúmenes asociados y levantó nuevamente el proceso, al realizar una petición se puede apreciar que si se reinicio el total de visitas esta vez.

```
vboxuser@ubuntu:~/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker$ sudo dock
er compose down -v
WARN[0000] /home/vboxuser/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker/do
cker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusi
on
[+] down 4/4
 ✔ Container nodo_web                  Removed                                                                  10.6s
 ✔ Container nodo_db                   Removed                                                                   0.9s
 ✔ Volume lab-so-docker_datos-redis    Removed                                                                   0.0s
 ✔ Network lab-so-docker_red-so-interna Removed                                                                  0.2s
vboxuser@ubuntu:~/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker$ sudo dock
er volume ls
DRIVER    VOLUME NAME
vboxuser@ubuntu:~/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker$
```

```
vboxuser@ubuntu:~/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker$ sudo dock
er compose up -d
WARN[0000] /home/vboxuser/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker/do
cker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusi
on
[+] up 4/4
 ✔ Network lab-so-docker_red-so-interna Created                                                                  0.1s
 ✔ Volume lab-so-docker_datos-redis    Created                                                                   0.1s
 ✔ Container nodo_db                   Started                                                                   1.5s
 ✔ Container nodo_web                  Started                                                                   1.5s
vboxuser@ubuntu:~/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker$ curl -s h
ttp://127.0.0.1:8080/
{"estado_db":"Conectado","hostname_contenedor":"b99a4c725f6c","mensaje":"Sistemas Operativos UCV - Contenerizacion Exi
tosa","total_visitas":1}
vboxuser@ubuntu:~/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker$
```

Paso 4. Inspección Forense de Namespaces desde el Kernel del Host

Ahora, debemos de realizar algunas inspecciones en el host para comprender que los contenedores son realmente procesos convencionales aislados y no máquinas virtuales como uno normalmente pensaría.

4.1. Primero queremos ver cómo un mismo proceso tiene un PID diferente dependiendo de si lo vemos desde adentro del contenedor o desde el host. ejecutamos docker exec nodo_db ps aux:

```
vboxuser@ubuntu:~/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker$ sudo dock
er exec nodo_db ps aux
PID   USER     TIME  COMMAND
    1 redis     0:01 redis-server *:6379
   30 root      0:00 ps aux
vboxuser@ubuntu:~/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker$
```

Para el contenedor, redis-server es el PID 1. Esto ocurre porque Docker inicia el contenedor con un namespace de PID aislado, donde el primer proceso que arranca recibe el número 1. Continuamos buscando ese mismo proceso en nuestra máquina virtual o el host con el comando ps -ef | grep redis-server:

```
vboxuser@ubuntu:~/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker$ ps -ef |
grep redis-server
dnsmasq    14799   14773  0 20:35 ?        00:00:01 redis-server *:6379
vboxuser   15178   11479  0 20:45 pts/1    00:00:00 grep --color=auto redis-server
vboxuser@ubuntu:~/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker$
```

El PID real en el host es un número alto (14799), no 1. Mientras que el contenedor ve PID 1, el host ve PID 14799. Demostrando asi que el kernel está traduciendo los PIDs gracias al namespace de PID.

4.2. Ahora vamos a ver los namespaces del proceso de Redis desde el host. Pero investigando vi que podemos guardar el PID del host en una variable para evitarnos la ladilla de tener que escribirlo a mano, y luego si Inspeccione los enlaces simbólicos de los namespaces de ese proceso con el comando sudo ls -l /proc/<PID_HOST>/ns/:

```
vboxuser@ubuntu:~/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker$ PID_HOST=
$(ps -ef | grep redis-server | grep -v grep | awk '{print $2}' | head -n 1)
echo $PID_HOST
14799
vboxuser@ubuntu:~/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker$ sudo ls -
l /proc/$PID_HOST/ns/
total 0
lrwxrwxrwx 1 dnsmasq vboxuser 0 Oct 11 20:43 cgroup -> 'cgroup:[4026532769]'
lrwxrwxrwx 1 dnsmasq vboxuser 0 Oct 11 20:43 ipc -> 'ipc:[4026532767]'
lrwxrwxrwx 1 dnsmasq vboxuser 0 Oct 11 20:35 mnt -> 'mnt:[4026532765]'
lrwxrwxrwx 1 dnsmasq vboxuser 0 Oct 11 20:35 net -> 'net:[4026532770]'
lrwxrwxrwx 1 dnsmasq vboxuser 0 Oct 11 20:43 pid -> 'pid:[4026532768]'
lrwxrwxrwx 1 dnsmasq vboxuser 0 Oct 11 20:50 pid_for_children -> 'pid:[4026532768]'
lrwxrwxrwx 1 dnsmasq vboxuser 0 Oct 11 20:43 time -> 'time:[4026532978]'
lrwxrwxrwx 1 dnsmasq vboxuser 0 Oct 11 20:50 time_for_children -> 'time:[4026532978]'
lrwxrwxrwx 1 dnsmasq vboxuser 0 Oct 11 20:35 user -> 'user:[4026531837]'
lrwxrwxrwx 1 dnsmasq vboxuser 0 Oct 11 20:43 uts -> 'uts:[4026532766]'
vboxuser@ubuntu:~/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker$
```

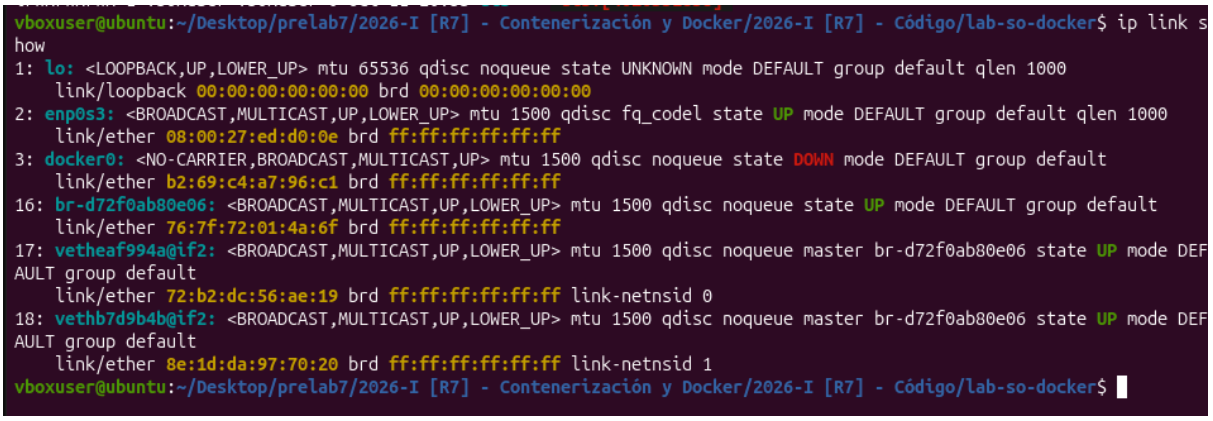
comparando con los namespaces de mi terminal

```
vboxuser@ubuntu:~/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker$ ls -l /pr
oc/$$/ns/
total 0
lrwxrwxrwx 1 vboxuser vboxuser 0 Oct 11 20:53 cgroup -> 'cgroup:[4026531835]'
lrwxrwxrwx 1 vboxuser vboxuser 0 Oct 11 20:53 ipc -> 'ipc:[4026531839]'
lrwxrwxrwx 1 vboxuser vboxuser 0 Oct 11 20:53 mnt -> 'mnt:[4026531832]'
lrwxrwxrwx 1 vboxuser vboxuser 0 Oct 11 20:53 net -> 'net:[4026531833]'
lrwxrwxrwx 1 vboxuser vboxuser 0 Oct 11 20:53 pid -> 'pid:[4026531836]'
lrwxrwxrwx 1 vboxuser vboxuser 0 Oct 11 20:53 pid_for_children -> 'pid:[4026531836]'
lrwxrwxrwx 1 vboxuser vboxuser 0 Oct 11 20:53 time -> 'time:[4026531834]'
lrwxrwxrwx 1 vboxuser vboxuser 0 Oct 11 20:53 time_for_children -> 'time:[4026531834]'
lrwxrwxrwx 1 vboxuser vboxuser 0 Oct 11 20:53 user -> 'user:[4026531837]'
lrwxrwxrwx 1 vboxuser vboxuser 0 Oct 11 20:53 uts -> 'uts:[4026531838]'
vboxuser@ubuntu:~/Desktop/prelab7/2026-I [R7] - Contenerización y Docker/2026-I [R7] - Código/lab-so-docker$
```

Los resultados evidencian que el contenedor posee namespaces completamente independientes para pid, net, mnt, ipc, uts, cgroup y time, lo que le otorga una vista aislada de los procesos, la red, el sistema de archivos y la comunicación entre procesos.

Sin embargo, el namespace user presenta el mismo inodo (4026531837) en ambos entornos. Esto demuestra que, por defecto, Docker no aplica remapeo de identificadores UID/GID, por lo que el usuario root dentro del contenedor se corresponde con el usuario root del host anfitrión.

4.3. Ahora debemos de hacer una inspección de la red virtual (veth pairs y Bridge). Vamos a ver cómo Docker conecta los contenedores a la red con el comando ip link show:



Se identificaron las siguientes interfaces:

1. docker0: Bridge por defecto de Docker, en estado NO-CARRIER porque los contenedores en este laboratorio utilizan una red personalizada.
2. br-d72f0ab80e06: Bridge virtual creado por docker-compose para la red red-so-interna. Actúa como un conmutador (switch) de software que interconecta los contenedores.
3. vethaea9944@if2 y veth7d9b4d4@if1: Pares de interfaces virtuales Ethernet (veth pairs). Cada contenedor tiene un extremo de este "cable virtual" en su propio namespace de red, mientras que el otro extremo está conectado al bridge br-d72f0ab80e06 (indicado por master br-d72f0ab80e06).

Esta arquitectura resalta que Docker no emula hardware de red, sino que utiliza mecanismos nativos del kernel de Linux como bridges y veth para proporcionar conectividad aislada. Con esta configuración, los contenedores nodo_web y nodo_db pueden resolverse por DNS interno y comunicarse a través de la red privada, mientras permanecen aislados de la red del host y de otros contenedores.

Ejemplos

En este prelab decidí hacer unos cuantos ejemplos para poder entender mejor los puntos investigados en el marco teórico, debo de aclarar que estos ejercicios la mayoria estan hechos por ia, con el objetivo de entender mejor las estructuras, aquí dejo el paso a paso para obtener el resultado de las imágenes mostradas en el marco teórico

6.1) Suponiendo que tenemos dos directorios:
/capa_base que contiene el archivo app.py.
/capa_superior contiene el archivo config.txt.

Al montar un UnionFS con estas dos capas, el punto de montaje /merged mostrará ambos archivos: app.py y config.txt, como si estuvieran en el mismo directorio, aunque físicamente residan en ubicaciones separadas. Creamos las dos capas de prueba

```
vboxuser@ubuntu:~/Desktop/prelab7$ mkdir -p ~/Desktop/prelab7/ejemplo-unionfs
cd ~/Desktop/prelab7/ejemplo-unionfs

# Capa base (solo lectura)
mkdir -p capa_base
echo "print('Hola desde app.py')" > capa_base/app.py

# Capa superior (lectura/escritura)
mkdir -p capa_superior
echo "config_key=valor" > capa_superior/config.txt

# Directorios de trabajo para OverlayFS
mkdir -p merged work upper
```

Verificamos

```
vboxuser@ubuntu:~/Desktop/prelab7/ejemplo-unionfs$ ls -R capa_base capa_superior
capa_base:
app.py

capa_superior:
config.txt
```

Montar OverlayFS manualmente

```
vboxuser@ubuntu:~/Desktop/prelab7/ejemplo-unionfs$ sudo mount -t overlay overlay \
  -o lowerdir=$(pwd)/capa_base,upperdir=$(pwd)/upper,workdir=$(pwd)/work \
  $(pwd)/merged
[sudo: authenticate] Password:
```

verificar que el montaje se realizó

```
vboxuser@ubuntu:~/Desktop/prelab7/ejemplo-unionfs$ mount | grep merged
overlay on /home/vboxuser/Desktop/prelab7/ejemplo-unionfs/merged type overlay (rw,relatime,lowerdir=/home/vboxuser/Des
ktop/prelab7/ejemplo-unionfs/capa_base,upperdir=/home/vboxuser/Desktop/prelab7/ejemplo-unionfs/upper,workdir=/home/vbo
xuser/Desktop/prelab7/ejemplo-unionfs/work,uuid=on,nouserxattr)
vboxuser@ubuntu:~/Desktop/prelab7/ejemplo-unionfs$
```

Ver el punto de montaje /merged

```
vboxuser@ubuntu:~/Desktop/prelab7/ejemplo-unionfs$ ls -la merged/
total 12
drwxrwxr-x 1 vboxuser vboxuser 4096 Oct 12 02:52 .
drwxrwxr-x 7 vboxuser vboxuser 4096 Oct 12 02:52 ..
-rw-rw-r-- 1 vboxuser vboxuser   27 Oct 12 02:52 app.py
vboxuser@ubuntu:~/Desktop/prelab7/ejemplo-unionfs$
```

6.2) Entonces, podemos verlo de la siguiente manera. Si dos contenedores se instancian desde la misma imagen python:3.11-alpine por ejemplo, ambos comparten la misma capa de solo lectura que contiene el intérprete de Python. Si el contenedor A modifica /etc/hosts, OverlayFS copia ese archivo a la capa de escritura del contenedor A. El contenedor B sigue viendo el /etc/hosts original de la imagen. El espacio en disco solo se incrementa por el archivo copiado, no por toda la imagen. Crear los dos contenedores desde la misma imagen, El comando sleep 3600 mantiene el contenedor vivo durante 1 hora para poder inspeccionar.

```
vboxuser@ubuntu:~/Desktop/prelab7/ejemplo-unionfs$ sudo docker run -d --name contenedor-A python:3.11-alpine sleep 360
0
sudo docker run -d --name contenedor-B python:3.11-alpine sleep 3600
34d172b57cbfbe4c34460d30eff243d81789dfde8b3fab6f81a87f251e75bb06
552d242d90b490d46a2f3f8d4f0c54e8dc311d995f28eaeca15d86af587750d2
```

Verifica que ambos están corriendo

```
vboxuser@ubuntu:~/Desktop/prelab7/ejemplo-unionfs$ sudo docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED          STATUS          PORTS
                    NAMES
552d242d90b4   python:3.11-alpine   "sleep 3600"             10 seconds ago   Up 9 seconds
                    contenedor-B
34d172b57cbf   python:3.11-alpine   "sleep 3600"             11 seconds ago   Up 10 seconds
                    contenedor-A
b7fe1935ee0b   lab-so-web:v1        "python app.py"          6 hours ago      Up 2 hours      0.0.0.0:8080->5000/tcp,
[::]:8080->5000/tcp   nodo_web
58bd5e1cb5f4   redis:alpine         "docker-entrypoint.s…"   6 hours ago      Up 2 hours      6379/tcp
                    nodo_db
```

6.3) vistazo a las capas del contenedor. Crear los archivos de la app

```
  GNU nano 8.7.1                                                app.py
print("Hola desde la app")
```

```
  GNU nano 8.7.1                                              config.txt
clave=valor-inicial
```

Crear el Dockerfile

```
  GNU nano 8.7.1                                        Dockerfile *
# Capa 1: base
FROM python:3.11-alpine

# Capa 2: copia de config.txt
COPY config.txt /etc/config.txt

# Capa 3: copia de app.py
COPY app.py /app/app.py

# Capa 4: comando de arranque
CMD ["python", "-c", "import time; print(open('/etc/config.txt').read()); time.sleep(3600)"]
```

Construir la imagen

```
vboxuser@ubuntu:~/Desktop/prelab7/demo-capas$ sudo docker build -t demo-capas:v1 .
[+] Building 3.9s (8/8) FINISHED                                                          docker:default
 => [internal] load build definition from Dockerfile                                               0.3s
 => => transferring dockerfile: 316B                                                               0.1s
 => [internal] load metadata for docker.io/library/python:3.11-alpine                              0.3s
 => [internal] load .dockerignore                                                                  0.2s
 => => transferring context: 2B                                                                    0.0s
 => CACHED [1/3] FROM docker.io/library/python:3.11-alpine@sha256:d9368b3a5ac59afea7b5d4f2e2aea0941dbf9fdee9c36  0.5s
 => => resolve docker.io/library/python:3.11-alpine@sha256:d9368b3a5ac59afea7b5d4f2e2aea0941dbf9fdee9c369c5bec0  0.5s
 => [internal] load build context                                                                  0.3s
 => => transferring context: 119B                                                                  0.0s
 => [2/3] COPY config.txt /etc/config.txt                                                          0.9s
 => [3/3] COPY app.py /app/app.py                                                                  0.4s
 => exporting to image                                                                             0.8s
 => => exporting layers                                                                            0.3s
 => => exporting manifest sha256:e2a4939c8a5b95e25d2d3b0999dcb36649f240d67def630dd7adad783c205fbb  0.0s
 => => exporting config sha256:48dcf9f61127914f052b6efa13cf0d7882cd88513e716a866ce9e687400609d5    0.0s
 => => exporting attestation manifest sha256:7ef86efb09b242cd4541cc6da2b37d5c2159f97bfba511a7c5377640be9904e3  0.0s
 => => exporting manifest list sha256:8d30120b253c49d5a99339e1b2bed99209c07c909c2eb6939f5f9e6dbcb0eb1d  0.0s
 => => naming to docker.io/library/demo-capas:v1                                                   0.0s
 => => unpacking to docker.io/library/demo-capas:v1                                                0.3s
vboxuser@ubuntu:~/Desktop/prelab7/demo-capas$
```

Ver el historial de capas de la imagen, cada instrucción del Dockerfile crea una capa. Las capas más antiguas (abajo) vienen de la imagen base. Las de arriba son las que agregue.

```
vboxuser@ubuntu:~/Desktop/prelab7/demo-capas$ sudo docker history demo-capas:v1
IMAGE          CREATED              CREATED BY                                      SIZE      COMMENT
8d30120b253c   About a minute ago   CMD ["python" "-c" "import time; print(open(…   0B        buildkit.dockerfile.v0
<missing>      About a minute ago   COPY app.py /app/app.py # buildkit              12.3kB    buildkit.dockerfile.v0
<missing>      About a minute ago   COPY config.txt /etc/config.txt # buildkit      12.3kB    buildkit.dockerfile.v0
<missing>      10 days ago          CMD ["python3"]                                 0B        buildkit.dockerfile.v0
<missing>      10 days ago          RUN /bin/sh -c set -eux;  for src in idle3 p…   16.4kB    buildkit.dockerfile.v0
<missing>      10 days ago          RUN /bin/sh -c set -eux;  apk add --no-cach…    57.9MB    buildkit.dockerfile.v0
<missing>      10 days ago          ENV PYTHON_SHA256=bfb74ad39efae27cda510f134a…   0B        buildkit.dockerfile.v0
<missing>      10 days ago          ENV PYTHON_VERSION=3.11.17                      0B        buildkit.dockerfile.v0
<missing>      10 days ago          ENV GPG_KEY=A035C8C19219BA821ECEA86B64E628F8…   0B        buildkit.dockerfile.v0
<missing>      10 days ago          RUN /bin/sh -c set -eux;  apk add --no-cache…   2.83MB    buildkit.dockerfile.v0
<missing>      10 days ago          ENV LANG=C.UTF-8                                0B        buildkit.dockerfile.v0
<missing>      10 days ago          ENV PATH=/usr/local/bin:/usr/local/sbin:/usr…   0B        buildkit.dockerfile.v0
<missing>      3 weeks ago          CMD ["/bin/sh"]                                 0B        buildkit.dockerfile.v0
<missing>      3 weeks ago          ADD alpine-minirootfs-3.24.2-x86_64.tar.gz /…   9.08MB    buildkit.dockerfile.v0
vboxuser@ubuntu:~/Desktop/prelab7/demo-capas$
```

Crear un contenedor a partir de la imagen y verificar que esta corriendo

```
vboxuser@ubuntu:~/Desktop/prelab7/demo-capas$ sudo docker run -d --name demo-capas demo-capas:v1
4be50ecbb884582a7a68ea1fc96ca3416c90311b6132183b378ce2f1e958e945
vboxuser@ubuntu:~/Desktop/prelab7/demo-capas$ sudo docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED          STATUS          PORTS
                  NAMES
4be50ecbb884   demo-capas:v1        "python -c 'import t…"   7 seconds ago    Up 6 seconds
                  demo-capas
552d242d90b4   python:3.11-alpine   "sleep 3600"             15 minutes ago   Up 15 minutes
                  contenedor-B
34d172b57cbf   python:3.11-alpine   "sleep 3600"             15 minutes ago   Up 15 minutes
                  contenedor-A
b7fe1935ee0b   lab-so-web:v1        "python app.py"          6 hours ago      Up 3 hours      0.0.0.0:8080->5000/tcp,
[::]:8080->5000/tcp   nodo_web
58bd5e1cb5f4   redis:alpine         "docker-entrypoint.s…"   6 hours ago      Up 3 hours      6379/tcp
                  nodo_db
vboxuser@ubuntu:~/Desktop/prelab7/demo-capas$
```

Inspeccionar las capas del contenedor, ahora obtenemos las rutas de las capas

```
vboxuser@ubuntu:~/Desktop/prelab7/demo-capas$ sudo docker inspect demo-capas --format='{{json .GraphDriver.Data}}' | p
ython3 -m json.tool
{
    "ID": "05cac6fc4f35dfaec73b2ce97edad1cb7801b3644116f60e309d824f5215f22d",
    "LowerDir": "/var/lib/docker/overlay2/f793f41b853c06a83ad425370ba7e6d45df5309f1131438300fe15bdfbab98fd-init/diff:/
var/lib/docker/overlay2/u42wchjo8hwtdd10ise62dkyl/diff:/var/lib/docker/overlay2/po1d7ejoe53hgsg48vknujcn9/diff:/var/li
b/docker/overlay2/7cac62c2ed72e3da6dcc9785583f6346e5fff329ab1255e1624220d471d42c4b/diff:/var/lib/docker/overlay2/f0124
84dbbd53db14487fa7971a2a1deafab8ade30dd781649b9656e31d91417/diff:/var/lib/docker/overlay2/2ce2dccc0f918905037bfea1b04a
a5dff73eef2ceb7f18684ccd74b98962e066/diff:/var/lib/docker/overlay2/0b94a4f23837fe97dcaa7bdbd7be0aed9d5bf1341cbcd580e60
c8dc174783a63/diff",
    "MergedDir": "/var/lib/docker/overlay2/f793f41b853c06a83ad425370ba7e6d45df5309f1131438300fe15bdfbab98fd/merged",
    "UpperDir": "/var/lib/docker/overlay2/f793f41b853c06a83ad425370ba7e6d45df5309f1131438300fe15bdfbab98fd/diff",
    "WorkDir": "/var/lib/docker/overlay2/f793f41b853c06a83ad425370ba7e6d45df5309f1131438300fe15bdfbab98fd/work"
}
vboxuser@ubuntu:~/Desktop/prelab7/demo-capas$
```

Bibliografías:

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley. (Capítulo 19: Networks and Distributed Systems - comunicación en red, NAT y virtualización de redes).

Stallings, W. (2018). Operating Systems: Internals and Design Principles (9th ed.). Pearson. (Capítulo 13: Distributed Processing, Client/Server, and Clusters - protocolos de interconexión y traducción de direcciones).

Docker Documentation. (s.f.). Packet filtering and firewalls. Recuperado de Packet filtering and firewalls | Docker Docs el 11/10/2026 (Consultado para las reglas iptables que Docker utiliza para NAT y publicación de puertos).

Phusion Blog. (2015, 20 de enero). Docker and the PID 1 zombie reaping problem. Recuperado de Docker and the PID 1 zombie reaping problem el 11/10/2026 (Consultado para el análisis clásico del problema de zombies en contenedores).

KodeKloud. (s.f.). Networking Deep Dive Docker. Recuperado de Networking Deep Dive Docker - KodeKloud el 11/10/2026 (Consultado para detalles sobre veth pairs, bridges y NAT en Docker).

KodeKloud. (s.f.). Docker Best Practices for Production Containers [PDF]. Recuperado de https://kodekloud.com/wp-content/uploads/2021/08/Docker-Certified-Associate.pdf el 11/10/2026 (Consultado para la discusión sobre la forma shell vs. exec y el manejo de señales).

NSFOCUS. (s.f.). Technical Report on Container Security (II)-3. Recuperado de Security Risks and Challenges Archives - NSFOCUS el 11/10/2026 (Consultado para el análisis de las tecnologías subyacentes de red en contenedores: namespaces, Linux bridges y veth pairs).

Linux Kernel Documentation. (s.f.). OverlayFS. Recuperado de https://www.kernel.org/doc/Documentation/filesystems/overlayfs.txt el 11/10/2026 (Consultado para la documentación oficial sobre el sistema de archivos de unión).

Docker Documentation. (s.f.). Use the OverlayFS storage driver. Recuperado de OverlayFS storage driver | Docker Docs el 11/10/2026 (Consultado para ejemplos prácticos de montaje OverlayFS con Docker).

LKML (Linux Kernel Mailing List). (2023, 1 de junio). Re: [PATCH] Documentation: Clarify usage of memory limits. Recuperado de https://lkml.org/lkml/2023/6/1/1294 el 11/10/2026 (Consultado para la discusión sobre memory.high vs. memory.max y la invocación del OOM Killer).

Docker Success Center. (s.f.). What causes a container to exit with code 137?. Recuperado de https://success.docker.com/api/articles/what-causes-a-container-to-exit-with-code-137/pdf

el 11/10/2026 (Consultado para la explicación del código 137 como resultado de kill -9 (128 + 9)).

Alibaba Cloud. (2026, 20 de septiembre). Troubleshoot the OOM Killer. Recuperado de OOM Killer causes and solutions in Alibaba Cloud Linux el 11/10/2026 (Consultado para causas y escenarios de activación del OOM Killer en cgroups).